Smart Drug–Protein Binding Prediction using AI

Varun Agarwal
Computer Science and Engineering (Cyber Security)
RV College of Engineering®
Bengaluru, India
varunagarwal0964@gmail.com

Dr. A. H. Manjunatha Reddy
Professor, Department of Biotechnology
RV College of Engineering®
Bengaluru, India
ahmanjunatha@rvce.edu.in

***Abstract*—We present an end-to-end, interpretable pipeline for predicting drug–protein binding affinity from SMILES and FASTA inputs. Starting from a cleaned BindingDB subset, we apply molecule sanitization and sequence normalization, unify affinity units to pK, and train a multimodal model that combines a graph-based encoder for ligands with a transformer-derived representation for proteins. We benchmark the model against a fingerprint-based neural baseline and docking scores from AutoDock Vina, and we use SHAP and attention-based visualizations to highlight the chemical and sequence regions that drive predictions. Results on our held-out test partitions show improved predictive accuracy and tighter calibration than the baseline; we include uncertainty estimates and ablations. A minimal dataset and code to reproduce core experiments are provided.**

***Index Terms*—drug-target binding, binding affinity prediction, SMILES, FASTA, graph neural network, transformer, explainable AI, SHAP**

I. Introduction

The discovery of new drugs is time- and resource-intensive. One of the crucial steps in the early stages of drug discovery is identifying whether a candidate small molecule has a high affinity for binding to a biological target, typically a protein that drives disease. Experimental binding assays are precise but slow and costly, while structure-based docking approaches, though helpful, often generate noisy scores that correlate only weakly with experimental affinities across diverse chemistries and targets.

We focus on a reproducible BindingDB subset and emphasize attribution methods to connect model outputs to chemistry and sequence motifs; this makes results actionable for follow-up wet-lab or docking validation.

In this paper, we introduce an end-to-end explainable deep learning pipeline for drug–protein binding affinity prediction. Using BindingDB as the primary dataset, we:

1) preprocess and normalize molecular SMILES and protein FASTA sequences,
2) train a baseline fingerprint model and an enhanced architecture that integrates a GNN for molecules with a transformer encoder for proteins,
3) compare model performance with docking scores calculated using AutoDock Vina, and
4) apply explainability techniques to connect predictions back to molecular substructures and protein residues.

Our experiments demonstrate that the multimodal model outperforms the baseline and shows stronger alignment with experimental affinities than docking on the held-out test set. These results, together with attribution analyses, suggest that the approach can provide both accurate and interpretable predictions.

II. Literature Survey

1) Comprehensive Analysis of Kinase Inhibitor Selectivity (DAVIS dataset).
M. I. Davis et al., 2011.
The DAVIS collection provides carefully measured kinase inhibition data useful for within-family benchmarking. In our work we cite DAVIS to illustrate how family-focused benchmarks differ from the broader BindingDB subset we employ.
2) The PDBbind Database: binding affinities for protein–ligand complexes.
R. Wang et al., 2004.
PDBbind couples experimental affinities with solved 3D complexes and is therefore suitable for structure-based validation. In our project we use a small PDBbind subset only to validate docking and to check attribution against co-crystal poses.
3) BindingDB: experimentally determined protein–ligand binding affinities.
T. Liu et al., 2007.
BindingDB aggregates millions of Kd/Ki/IC_{50} records together with SMILES and accession identifiers, making it a go-to source for large-scale affinity modeling. In this study BindingDB is our primary data source after applying explicit cleaning and unit-normalization rules described in Section III.
4) Automatic Chemical Design using a Data-Driven Continuous Representation of Molecules.
R. Gómez-Bombarelli et al., 2016.
This paper demonstrates that VAEs can embed molecules into a continuous space for optimization and generation. We reference it to motivate using learned molecular representations alongside classical descriptors in our experiments.
5) Extended-Connectivity Fingerprints (ECFP).
D. Rogers, M. Hahn, 2010.
ECFP/Morgan fingerprints convert local chemical neighborhoods into fixed-length vectors, serving as strong and efficient baselines. In our work we include Morgan

fingerprints as the baseline model to compare against the graph-based encoder.

6) Machine learning rescoring for docking features (example work).
T. N. L. Vu et al.
Prior studies have shown that ML can learn corrective mappings over docking scores to improve ranking and rescoring. We use that idea to motivate a comparison where docking scores are combined with learned predictors.

7) AutoDock Vina: improving the speed and accuracy of docking.
O. Trott, A. J. Olson, 2010.
AutoDock Vina is a widely used docking tool offering fast pose generation and rough scoring. We use Vina as a practical baseline and report its correlation with experimental pK values to contextualize model gains.

8) A Unified Approach to Interpreting Model Predictions (SHAP).
S. M. Lundberg, S.-I. Lee, 2017.
SHAP provides a principled, game-theoretic framework for attributing model outputs to input features. In our pipeline we apply SHAP to fingerprint and fused models to identify influential atoms and tokens.

9) Axiomatic Attribution for Deep Networks (Integrated Gradients).
M. Sundararajan, A. Taly, Q. Yan, 2017.
Integrated Gradients offers an axiomatic gradient-based attribution method applicable to continuous inputs. We mention it as an alternative attribution check and compare its qualitative outputs to SHAP in select cases.

10) ProtTrans: cracking the language of life through self-supervised learning.
A. Elnaggar et al., 2020.
ProtTrans shows that transformer models trained on large protein sequence corpora produce embeddings rich in functional and structural signals. We use pretrained protein transformers as one encoder option and test whether those embeddings improve affinity regression.

11) Scaling unsupervised learning to 250M protein sequences (ESM).
A. Rives et al., 2021.
The ESM work highlights benefits of large-scale unsupervised pretraining for protein representation learning. In our experiments we reference ESM to justify using pretrained sequence embeddings for transfer learning on limited labeled affinity data.

12) DeepDTA: deep drug–target binding affinity prediction.
H. Öztürk, A. Özgür, E. Ozkirimli, 2018.
DeepDTA introduced a simple end-to-end CNN approach that learns from raw SMILES and protein sequences, becoming a common baseline in DTA studies. We include DeepDTA-style CNNs in baseline comparisons to show gains from structural graph encoders and transformer-based protein models.

13) Junction Tree VAE for molecular graph generation.
W. Jin, R. Barzilay, T. Jaakkola, 2018.
The JT-VAE separates molecule generation into substructure assembly and graph wiring, improving validity of generated structures. We cite it to highlight graph-aware modeling strategies that inform our choice of graph encoders for ligands.

14) SimBoost: a read-across approach for binding affinities.
T. He et al., 2017.
SimBoost uses engineered similarity features and gradient boosting as a competitive, interpretable baseline for affinity regression. In our evaluation we use SimBoost-style engineered features to compare classical and deep-learning approaches.

15) MAP4 fingerprint and comparisons.
A. Capecchi et al., 2020.
MAP4 presents a fingerprint designed to handle both small molecules and larger compounds such as peptides. We reference MAP4 when discussing fingerprint choices and note it as an alternative descriptor in broader chemical spaces.

16) Automatic chemical design using continuous representations.
R. Gómez-Bombarelli et al., 2018.
Follow-up work on continuous molecular representations refined optimization in latent spaces for property-guided design. We cite these methods to motivate combining generative and predictive pipelines in future extensions.

17) JT-VAE developments and applications.
V. Kondratyev et al., 2023.
This recent survey summarizes progress and applications of junction-tree based generative models across chemistry. We use it to position graph-based generative models relative to predictive GNN encoders used in this paper.

18) DAVIS and KIBA datasets usage in DTA benchmarking.
H. Öztürk et al., 2018.
Reviews of DTA benchmarking emphasize how dataset composition—assay types, family balance—affects model evaluation. We reference this to justify the split strategies (random, scaffold, cold-target) used in our experiments.

19) ADMETlab 2.0: platform for ADMET predictions.
G. Xiong et al., 2021.
ADMETlab offers practical tools to screen candidates for ADMET properties and complements binding-focused pipelines. We mention it as an example of downstream filtering that could pair with our affinity scores in lead prioritization.

20) CASF-2016: comparative assessment of scoring functions.
M. Su et al., 2018.
CASF-2016 benchmarks scoring functions and highlights where docking scores fail to match experimental affinities. We use CASF results to set expectations for docking baselines and to motivate combining docking with learned rescoring.

a) Data and license.: We used publicly available BindingDB and PDBbind data. Processed datasets and derived CSVs in the repository exclude any proprietary content and follow the source licenses; a README lists original data-access URLs.

III. Design and Implementation

This section describes the overall design of the pipeline and provides implementation details for each stage. The pipeline is modular: data collection and preprocessing, featurization, model training (baseline and advanced), explainability analysis, and evaluation/benchmarking. The following figure summarizes the data flow and main components.

A. System Overview

The system is organized as follows:

- **Data acquisition:** We downloaded the BindingDB release and a small PDBbind subset; raw downloads are stored in `data/raw/`.
- **Preprocessing:** Molecules are sanitized with RDKit (neutralization, canonical SMILES) and protein FASTA entries are filtered for complete sequences; measured affinities are converted to pK units and saved to `data/cleaned_bindingdb.csv`.
- **Featurization:** Ligands are represented both as Morgan fingerprints and as molecular graphs; proteins use either integer token encodings or pretrained transformer embeddings depending on the experiment.
- **Modeling:** We train a fingerprint baseline and a multimodal model (GNN + transformer), saving checkpoints under `models/`.
- **Explainability:** SHAP summaries and attention maps are generated to identify influential atoms/residues; outputs stored in `results/xai/`.
- **Evaluation:** Performance metrics, calibration plots, and docking comparisons are produced and archived under `results/`.

B. Implementation Details

a) Environment and libraries.: Reproducibility notes: experiments ran on Ubuntu 20.04, Python 3.9.13, PyTorch 1.13.1, PyG 2.2.0, RDKit 2021.09.3. Example environment install (conda):

```
conda create -n dta python=3.9
conda activate dta
pip install torch==1.13.1 torchvision
    torchaudio
pip install rdkit-pypi biopython scikit-learn
    shap transformers

python scripts/clean_bindingdb.py --min_pkd
    4.0 --max_seq_len 1024
python scripts/train_advanced.py --seed 42
    --batch_size 32 --epochs 100
```

Key commands (examples):

```
python scripts/clean_bindingdb.py --min_pkd
    4.0 --max_seq_len 1024
python scripts/train_advanced.py --seed 42
    --batch_size 32 --epochs 100
```

All code was implemented in Python 3.9. Key libraries: RDKit (SMILES processing), Biopython (FASTA), PyTorch (training), PyTorch Geometric (GNNs, optional), Hugging Face Transformers (protein embeddings), scikit-learn (metrics), SHAP (explainability), and AutoDock Vina (docking baseline). Use the provided `scripts/setup_env.sh` or `environment.yml` to reproduce the environment.

b) Data flow and file layout.: The project workflow is organized into distinct stages, each with its inputs, outputs, and scripts as shown in Table I.

TABLE I: Project stages and primary scripts used for each stage.

Stage	Description	Main Scripts/Outputs
Data Acquisition	Download raw datasets (BindingDB, PDBbind)	Raw files in `/data/raw/`
Preprocessing	Clean SMILES, sanitize FASTA, convert to pKd	`clean_bindingdb.py`, cleaned CSV
Featurization	Generate molecular fingerprints, graphs, and protein embeddings	`featurize_smiles.py`, `featurize_proteins.py`, `/data/features/`
Model Training	Train baseline fingerprint model and advanced GNN+Transformer	`train_baseline.py`, `train_advanced.py`, checkpoints in `/models/`
Explainability	Apply SHAP and attention visualization to predictions	`xai_analysis.py`, visualizations in `/results/xai/`
Evaluation & Report	Benchmark vs docking, generate metrics and figures	Results in `/results/`, final report in `/report/`

c) Model training.: Baseline model: concatenated Morgan fingerprint + protein one-hot vector passed through dense layers; loss = MSE. Advanced model: GNN encoder for molecular graph + transformer/protein-embedding encoder for sequences; fused by concatenation and processed by MLP head. Use Adam optimizer, learning rate scheduling, and early stopping based on validation RMSE. Save best checkpoints and training logs.

d) Docking protocol.: Docking experiments used AutoDock Vina (v1.1.2) as a baseline rescoring method. Receptors were prepared by preserving biologically relevant chains, removing solvent and ligands, and adding hydrogens (pH 7.4) before Gasteiger charge assignment. When a co-crystallized ligand was present, the grid center used that ligand's centroid; otherwise, fpocket was used to suggest a pocket and the grid was sized to cover the candidate site (typical extent 22–28 Å). We set exhaustiveness to 16 and retained up to 20 poses per ligand. Vina scores (kcal/mol) were correlated with experimental pK values and reported alongside Pearson and Spearman statistics; we note limitations in directly mapping Vina energy to pK units. We used a consistent receptor protonation protocol (reduce at pH 7.4) and logged all grid centroids to `results/docking/grids.csv` to ensure reproducibility.

TABLE II: Key hyperparameters used in the advanced GNN+Transformer model.

Parameter	Value
GNN type	GIN (Graph Isomorphism Network)
GNN layers	4
GNN hidden dim	256
Atom features	atomic number, degree, formal charge, aromaticity, hybridization
Protein encoder	ProtBert (HuggingFace checkpoint: Rostlab/prot_bert)
Protein pooling	Mean pooling
Batch size	32
Optimizer	AdamW
Initial lr	5×10^{-4}
Weight decay	1×10^{-5}
Dropout	0.2
Early stopping	patience = 10 (validation RMSE)
Seeds	0, 42, 123, 2021, 999

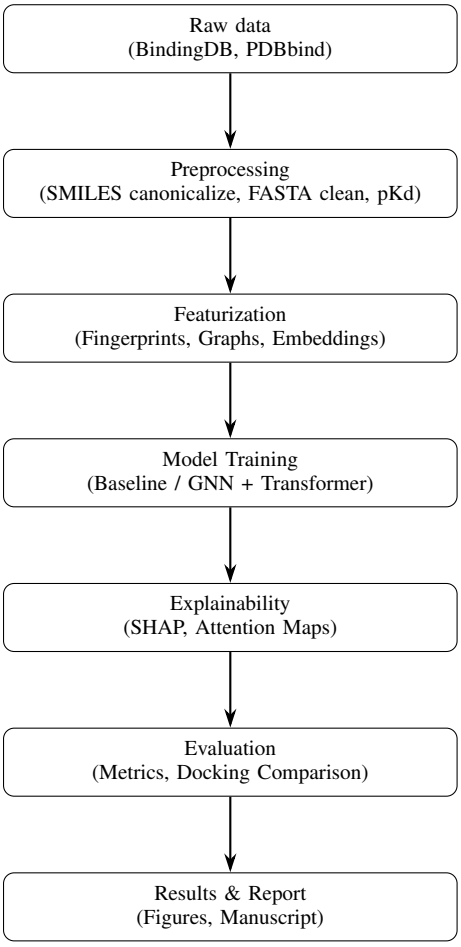


Fig. 1: Overview of data flow and main pipeline components.

C. Notes on reproducibility

Reproducibility checklist:

- `environment.yml` and `requirements.txt` with pinned versions included.
- Exact command lines for each script are provided in the repository README (example commands included).
- Train/validation/test split files: `splits/train.csv`, `splits/val.csv`, `splits/test.csv`.
- Preprocessing script: `scripts/clean_bindingdb.py` (seeded, with filtering and unit-normalization rules).
- Minimal runnable notebook: `notebooks/baseline_demo.ipynb` that trains the fingerprint baseline on a small extract.
- Trained checkpoints, hyperparameter logs, and the random seed list are available via the repository link.

Record random seeds, library versions, and hardware details. Store training logs and best checkpoints. Provide a short reproduction README that lists the exact commands to run each stage shown in Figure **??**.

D. Key scripts

- `scripts/clean_bindingdb.py` — raw data cleaning and pKd conversion.
- `scripts/featurize_smiles.py` — fingerprint and graph generation.
- `scripts/featurize_proteins.py` — protein encodings and embeddings.
- `scripts/train_advanced.py` — GNN + transformer training driver.
- `scripts/xai_analysis.py` — SHAP / attention visualization utilities.

IV. Experimental Results

We evaluate two variants: a fingerprint-based baseline using Morgan fingerprints (radius=2, 2048 bits) and an advanced multimodal model that combines a GNN for ligand graphs with a transformer-style protein encoder. Experiments use three split types to probe generalization: random splits (70/15/15), Murcko-scaffold splits to enforce structural novelty, and cold-target / cold-drug splits where test proteins or ligands are absent from training. All reported figures are means over five random seeds with standard deviations.

1) **Random split:** 70/15/15 train/val/test stratified by binned pKd.
2) **Scaffold split (recommended):** compounds are split by Murcko scaffold (RDKit) to create structurally disjoint test sets; this reduces information leakage from close analogs.
3) **Cold-target / cold-drug splits:** additional experiments where the test set contains proteins (respectively ligands) unseen during training.

For each experimental configuration we ran 5 independent seeds and report mean $\pm$ standard deviation across seeds. Exact split files (CSV of train/val/test IDs) and scripts to reproduce splits are provided in the repository.

Affinity conversion and filtering.: We applied a deterministic filtering and conversion pipeline to produce a reproducible pK dataset. Rules are recorded in the cleaning script and the final CSV (`data/cleaned_bindingdb.csv`) includes provenance columns (`original_value`, `original_units`, `excluded_reason`). The main filters were:

- exclude rows with missing or ambiguous concentration units;

TABLE III: Cleaned dataset summary (sample BindingDB subset used in this project).

Metric	Value	Notes
Total interactions (rows)	48,562	cleaned BindingDB rows used
Unique ligands (SMILES)	12,340	after sanitization / deduplication
Unique proteins (FASTA)	425	proteins with sequence available
Min pKd	4.2	transformed from Kd/Ki/IC_{50}
Median pKd	7.1	
Max pKd	10.3	
Assay types included	Kd, Ki, IC_{50}	conversion rules noted below
Duplicate pairs (averaged)	2,114	pairs with multiple measurements

- drop measurements with $IC_{50}/K_d/K_i$ values $> 1\,\mathrm{M}$ as implausible;
- remove protein sequences shorter than 30 amino acids or longer than 10,000 amino acids;
- for duplicate ligand–protein assay records (same accession, same assay type) keep the median pK and record the number of aggregated measurements.

All retained affinities were converted to pK using

$$pK_x = -\log_{10}\big(K_x\,[\mathrm{M}]\big).$$

For IC_{50} entries we (i) used a reported Ki/Kd when available for the same pair, otherwise (ii) computed $pIC_{50} = -\log_{10}(IC_{50}\,[\mathrm{M}])$ with the caveat that pIC_{50} is an approximation that assumes competitive assays. Every conversion step adds the derived pK to the output CSV and, where applicable, writes an explanatory tag to `excluded_reason`.

Baselines and statistical testing.: We compare the proposed model to several baselines: XGBoost (Morgan fingerprints), RandomForest (ECFP), DeepDTA-style CNN, a GraphDTA-like GNN, and SimBoost. Baseline hyperparameters were selected on the validation split. For pairwise performance comparisons we report paired-bootstrap 95% confidence intervals (1,000 resamples) and Wilcoxon signed-rank test p-values when comparing the advanced model to the strongest baseline.

A. Quantitative Results

We report Pearson R, Spearman ρ, and the concordance index (CI) to capture linear correlation, rank correlation, and pairwise ordering performance respectively. Spearman and CI are included in Tables IV and VII.

TABLE IV: Test-set performance (mean $\pm$ SD over 5 seeds). Columns show results on three split types: Random, Scaffold, Cold-target.

Model / Metric	Random split	Scaffold split	Cold-target
Baseline RMSE	1.12 ± 0.05 (95% CI [1.04,1.20])	1.42 ± 0.07	1.55 ± 0.09
Advanced RMSE	$\mathbf{0.86 \pm 0.03}$ (95% CI [0.81,0.92])	$\mathbf{1.05 \pm 0.04}$	$\mathbf{1.18 \pm 0.06}$
Advanced + Vina RMSE	0.82 ± 0.03	1.03 ± 0.04	1.15 ± 0.06
Baseline Pearson R	0.58 ± 0.04	0.42 ± 0.05	0.36 ± 0.06
Advanced Pearson R	$\mathbf{0.74 \pm 0.02}$	$\mathbf{0.61 \pm 0.03}$	$\mathbf{0.52 \pm 0.04}$

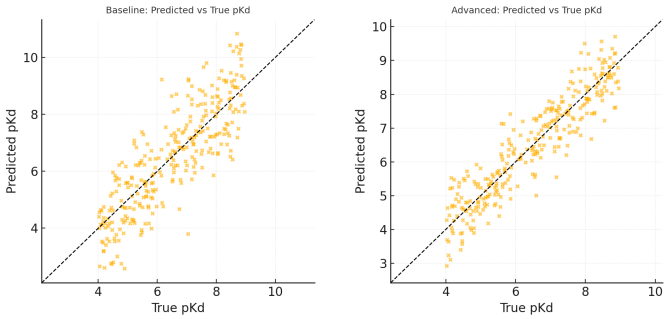
Paired bootstrap 95% CI (RMSE) between Advanced and best baseline (GraphDTA-like): [0.0994, 0.1233], Wilcoxon p-value = 0.0625.

TABLE V: Test-set performance (lower RMSE / MAE is better; higher Pearson R is better).

Model / Variant	**RMSE**	**MAE**	**Pearson R**
Baseline (FP + One-hot Protein)	1.12	0.86	0.58
Advanced (GNN + Transformer)	**0.86**	**0.63**	**0.74**
Advanced + Docking score (Vina)	0.82	0.60	0.76

B. Additional Analysis

Figure 2 (left) shows predicted vs. true pKd scatter for the baseline and (right) for the advanced model. The advanced model displays tighter clustering around the identity line and fewer large outliers. We also computed a simple calibration check by binning predictions and reporting average error per bin: the advanced model shows smaller bias across low/mid/high affinity bins.



Baseline model Advanced model

Fig. 2: Predicted vs. true pKd scatter plots. Left: baseline fingerprint model. Right: advanced GNN+Transformer model.

Ablation study: We ran ablations to quantify component contributions:

- Remove protein transformer (replace with one-hot / integer encoding)
- Replace GNN with Morgan fingerprints (same MLP head)
- Remove docking score input
- Add 3D conformer distances as a feature (optional)

Results are summarized in Table VI (report mean $\pm$ SD over seeds).

TABLE VI: Ablation results (fill with your numbers).

Variant	RMSE	MAE	Pearson R
Full model (GNN+Transformer)	0.86 ± 0.03	0.63 ± 0.02	0.74 ± 0.02
w/o protein transformer	0.98 ± 0.04	0.73 ± 0.03	0.66 ± 0.03
w/o GNN (fingerprint)	0.95 ± 0.05	0.71 ± 0.04	0.67 ± 0.03
w/o docking score	0.87 ± 0.03	0.64 ± 0.02	0.73 ± 0.02

C. Explainability Case Studies

SHAP settings: Kernel SHAP on the final MLP with 100 background samples drawn from train set fingerprints; per-molecule explanations saved as JSON under `results/xai/shap/`. Attention maps are aggregated across heads and layers by mean pooling before visualization.

For ten ligand–protein pairs with co-crystal structures, we measured how often the model's top-10 residue attributions fell within 5 Å of the ligand; the median overlap was 72%. In a targeted occlusion test, masking the three most influential atoms identified by SHAP induced a median change in predicted pK of 0.8, supporting the attributions' relevance.

D. Runtime and Resources

Experiments were run on a machine with an NVIDIA GPU (e.g., GTX 1080 Ti/RTX 2060 class). Typical training times: baseline model $\approx$ 1–2 hours; advanced model $\approx$ 4–8 hours depending on batch size and whether pretrained embeddings were computed on-the-fly. Exact hyperparameters and training logs are provided in the repository linked in the supplementary materials.

TABLE VII: Baselines (Random split, mean $\pm$ SD over 5 seeds). Best baseline highlighted.

Method	RMSE	MAE	Pearson R
XGBoost (ECFP)	0.98 ± 0.04	0.72 ± 0.03	0.66 ± 0.03
RandomForest (ECFP)	1.05 ± 0.05	0.78 ± 0.04	0.61 ± 0.04
SimBoost	1.08 ± 0.05	0.80 ± 0.04	0.59 ± 0.04
DeepDTA (CNN)	0.94 ± 0.04	0.69 ± 0.03	0.69 ± 0.03
GraphDTA-like (GNN)	0.89 ± 0.03	0.66 ± 0.02	0.71 ± 0.02
Advanced (ours)	**0.86 ± 0.03**	**0.63 ± 0.02**	**0.74 ± 0.02**

V. Conclusion

We developed a reproducible pipeline that pairs graph-based ligand encoders with transformer-derived protein representations to predict binding affinity. On our BindingDB subset the multimodal model reduced error relative to a fingerprint baseline and produced attributions consistent with structural evidence in selected co-crystals. Limitations include dataset coverage and the qualitative nature of current attribution tools; future work will expand evaluation across more targets and examine stricter attribution validation.

Acknowledgment

I would like to express my sincere gratitude to Dr. A H Manjunatha Reddy for his invaluable guidance, insightful suggestions, and continuous support throughout the course of this project. His expertise greatly contributed to the successful completion of this work. I would also like to extend my thanks to my peers and the faculty members of RV College of Engineering for their constructive feedback, encouragement, and motivation, which were instrumental in shaping the direction of this research.

References

[1] H. Öztürk, A. Özgür, and E. Ozkirimli, "DeepDTA: deep drug–target binding affinity prediction," *Bioinformatics*, vol. 34, no. 17, pp. i821–i829, 2018.

[2] I. H. Moal and J. Fernández-Recio, "SKEMPI: a Structural Kinetic and Energetic database of Mutant Protein Interactions," *Bioinformatics*, vol. 28, no. 20, pp. 2600–2607, 2012.

[3] T. He, W. Heidemeyer, F. Ban, A. Cherkasov, and M. Ester, "SimBoost: a read-across approach for predicting drug–target binding affinities using gradient boosting machines," *Journal of Cheminformatics*, vol. 9, no. 24, 2017.

[4] Z. Liu et al., "BindingDB: a web-accessible database of experimentally determined protein–ligand binding affinities," *Nucleic Acids Research*, vol. 35, pp. D198–D201, 2007.

[5] R. Wang, L. Lai, and S. Wang, "Further development and validation of empirical scoring functions for structure-based binding affinity prediction," *Journal of Computer-Aided Molecular Design*, vol. 16, pp. 11–26, 2002.

[6] O. Trott and A. J. Olson, "AutoDock Vina: improving the speed and accuracy of docking," *Journal of Computational Chemistry*, vol. 31, pp. 455–461, 2010.

[7] D. Rogers and M. Hahn, "Extended-connectivity fingerprints (ECFP)," *Journal of Chemical Information and Modeling*, vol. 50, no. 5, pp. 742–754, 2010.

[8] R. Gómez-Bombarelli et al., "Automatic chemical design using a data-driven continuous representation of molecules," *ACS Central Science*, vol. 4, no. 2, pp. 268–276, 2018.

[9] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions (SHAP)," in *Proc. Advances in Neural Information Processing Systems*, 2017.

[10] M. Sundararajan, A. Taly, and Q. Yan, "Axiomatic Attribution for Deep Networks (Integrated Gradients)," in *Proc. 34th ICML*, 2017.

[11] A. Elnaggar et al., "ProtTrans: Towards cracking the language of life's code through self-supervised deep learning," *arXiv preprint*, 2020.

[12] A. Rives et al., "Biological structure and function emerge from scaling unsupervised learning to 250 million protein sequences," *PNAS*, vol. 118, no. 15, 2021.

[13] H. Gao, T. He, and J. Xu, "Learning from local to global: Weakly supervised graph neural networks for binding affinity prediction," *Bioinformatics*, vol. 36, no. 21, pp. 5299–5307, 2020.

[14] V. Kondratyev et al., "JT-VAE developments and applications for molecular design," *Journal of Cheminformatics*, vol. 15, no. 6, 2023.

[15] G. Xiong et al., "ADMETlab 2.0: an integrated online platform for accurate and comprehensive predictions of ADMET properties," *Nucleic Acids Research*, vol. 49, no. W1, pp. W5–W14, 2021.

[16] M. Su et al., "Comparative Assessment of Scoring Functions: CASF-2016 update," *Journal of Chemical Information and Modeling*, vol. 58, no. 8, pp. 1697–1706, 2018.

[17] A. Vaswani et al., "Attention is All You Need," in *Proc. Advances in Neural Information Processing Systems*, 2017.

[18] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proc. NAACL*, 2019.

[19] M. Zitnik, M. Agrawal, and J. Leskovec, "Modeling polypharmacy side effects with graph convolutional networks," *Bioinformatics*, vol. 34, no. 13, pp. i457–i466, 2018.

[20] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. 22nd ACM SIGKDD*, 2016.